# MÉTRICAS DE SOFTWARE EN LOS DIFERENTES ENFOQUES DE GESTIÓN

Tema importante para el examen final.

Las métricas son valores que obtenemos de la realidad para conocer qué está pasando en un proyecto, proceso o producto de software. Permiten evaluar su estado o salud y tomar decisiones a partir de información concreta.

Son un input importante para las actividades de monitoreo y control. Si no medimos, no tenemos forma de conocer objetivamente los desvíos ni de evaluar si las acciones correctivas funcionaron.

IMPORTANTE: poner siempre la métrica en contexto. No mezclar enfoques ni conceptos. Cada enfoque de gestión tiene un propósito diferente y, por lo tanto, también cambia qué necesitamos medir y para qué usamos esa información.

La pregunta no es solamente qué métrica se utiliza, sino qué información aporta, a quién le sirve y qué decisión permite tomar.

## MÉTRICAS DE SOFTWARE EN EL ENFOQUE TRADICIONAL

Se basa en procesos definidos, donde se establece una planificación inicial a partir del alcance. Una vez definido qué se va a construir, se derivan los tiempos, esfuerzos y costos necesarios.

Se relaciona con la triple restricción: alcance, tiempo y costo. Las métricas permiten conocer cómo avanzamos respecto del plan y detectar desviaciones. También hay que considerar la calidad y otras restricciones, porque ajustar una variable puede afectar a las demás.

El plan no permanece necesariamente igual durante toda la vida del proyecto. Se va revisando y ajustando, por eso necesitamos comparar periódicamente lo previsto con lo real.

## DOMINIOS DE LAS MÉTRICAS

Es el ámbito en el que se aplica la métrica. Se distinguen tres:

- Producto: mide características del software que se está construyendo o que ya se construyó. Ejemplos: tamaño, líneas de código, requisitos, cantidad de defectos, cobertura de pruebas.

- Proyecto: permite evaluar cómo está avanzando un proyecto particular respecto de su planificación. Tiene sentido durante la vida de ese proyecto. Ejemplos: esfuerzo consumido, duración real frente a la estimada, cumplimiento de hitos y presupuesto.

- Proceso: permite conocer y mejorar cómo se trabaja dentro de la organización. No se limita a un único proyecto; las métricas de distintos proyectos pueden consolidarse para analizar el funcionamiento de los procesos de desarrollo.

Aclaración: una cosa es conocer cómo va un proyecto concreto y otra es evaluar cómo funcionan los procesos en la organización. No confundir proyecto con proceso.

MÉTRICAS BÁSICAS Y FUNDAMENTALES
Son cuatro:

1. Tamaño del producto: indica la magnitud del software. Puede expresarse en líneas de código, requisitos, casos de uso, funcionalidades, etc. Corresponde al ámbito del producto.

2. Esfuerzo: representa el trabajo humano invertido en una actividad o proyecto. Por ejemplo, horas-persona. Corresponde a proyecto y proceso. No es lo mismo esfuerzo que duración calendario.

3. Tiempo (calendario): es el tiempo transcurrido desde que comienza hasta que termina una actividad o proyecto. Corresponde a proyecto y proceso. Se compara lo estimado con lo realmente ocurrido.

4. Defectos: permite conocer problemas encontrados en el producto, su cantidad y sus características. Corresponde al ámbito del producto.

También menciona costos y presupuesto al comparar lo planificado con la ejecución real.

¿QUIÉN MIRA LAS MÉTRICAS Y PARA QUÉ?
No todos necesitan mirar exactamente la misma información. Depende de las responsabilidades y de las decisiones que tenga que tomar cada rol.

Ejemplos de la diapositiva «El sueño del pibe»:

- Desarrollador: esfuerzo de su tarea, esfuerzo y duración estimados y reales, cobertura de pruebas unitarias, cantidad y tipo de defectos encontrados en unit testing y en revisiones por pares.

- Equipo de desarrollo: tamaño del producto, duración entre hitos, cantidad de tareas planificadas y terminadas, distribución del esfuerzo, estado y cambios de los requerimientos, defectos encontrados durante integración y pruebas del sistema, porcentaje de pruebas ejecutadas.

- Organización: tiempo calendario, esfuerzo real frente al planificado, presupuesto real frente al previsto, precisión de las estimaciones y defectos que aparecen en las versiones liberadas.

La idea es que cada uno pueda acceder a información útil para su trabajo, no generar números sin un propósito concreto.

MANTENER LAS MÉTRICAS SIMPLES

Medir también consume esfuerzo. La toma de datos debería requerir el menor trabajo posible. Si podemos automatizar una medición, mejor.

La diapositiva usa una comparación: si estamos a kilómetros de nuestro destino, no tiene sentido medir la distancia en milímetros. Hay que elegir un nivel de precisión razonable según lo que necesitamos saber.

**Antes de decidir utilizar una métrica hay que preguntarse:**

**1. ¿Nos da más información de la que ya tenemos?**
**2. ¿Esa información tiene un beneficio práctico?**
**3. ¿Nos dice lo que realmente queremos saber?**

Si no aporta información nueva o no permite tomar decisiones, no tiene demasiado sentido medirla.

También importa cada cuánto se mide. La frecuencia debe permitir detectar el problema a tiempo para corregirlo. Si se toma una métrica cuando ya no podemos actuar, pierde parte de su utilidad.

IMPORTANTE: no se trata de usar métricas para juzgar el rendimiento individual de las personas, sino para entender y mejorar el trabajo.

Las métricas ayudan a buscar un equilibrio entre alcance, tiempo, costos, necesidades, expectativas y calidad.

MÉTRICAS DE SOFTWARE EN AMBIENTES ÁGILES

Se relacionan con procesos empíricos. A diferencia de un enfoque centrado en comparar permanentemente contra un plan fijo, aquí interesa observar resultados reales, inspeccionar y adaptar.

Las métricas siguen los principios del Manifiesto Ágil. La idea central es entregar software que funcione y que aporte valor al cliente.

Dos principios que guían la elección de las métricas:

- La mayor prioridad es satisfacer al cliente mediante entregas tempranas y continuas de software valioso y funcionando.
- El software funcionando es la principal medida de progreso.

Regla de oro: medir lo necesario y nada más. La medición debería surgir del trabajo y no transformarse en una actividad que agregue desperdicio.

No alcanza con decir que se trabajaron muchas horas o se hicieron muchas tareas. Lo más importante es qué software realmente funciona y fue entregado.

MÉTRICAS ÁGILES PRINCIPALES

## VELOCIDAD (VELOCITY)

Mide los Story Points correspondientes a las historias completadas y aceptadas en un Sprint.

Sirve para conocer el ritmo del equipo y para ayudar a planificar los siguientes Sprints a partir de resultados anteriores.

La velocidad se observa al finalizar el Sprint. Se busca que, con el tiempo, el equipo pueda alcanzar un ritmo relativamente estable; no significa que todos los Sprints deban dar exactamente el mismo número.

No contar historias que quedaron a medio terminar. Tampoco utilizar la velocidad de un equipo para juzgar o comparar a otro, porque las estimaciones y las condiciones de trabajo son propias de cada equipo.

## CAPACIDAD (CAPACITY)

Se estima al planificar el Sprint, durante la Sprint Planning.

Responde a la pregunta: ¿cuánto trabajo puede asumir este equipo en el próximo Sprint, dadas las personas y el tiempo realmente disponibles?

Hay que considerar cuántos integrantes trabajarán, ausencias, dedicación y otras actividades que restan tiempo.

En los primeros Sprints, cuando todavía no hay datos históricos suficientes, se puede partir de las horas disponibles para estimar el trabajo que es razonable asumir.

Aclaración importante: capacidad y velocidad no son lo mismo. Capacidad se utiliza para planificar a partir de la disponibilidad; velocidad expresa lo que efectivamente se completó y aceptó en Sprints anteriores.

## RUNNING TESTED FEATURES (RTF)

Cantidad de funcionalidades o historias que están implementadas, probadas y funcionando, de acuerdo con las condiciones de entrega consideradas en clase.
→ funcionalidades entregadas, testeadas y desplegadas en producción. Se revisa en la Sprint Review.

Es útil porque pone la atención en lo que realmente funciona, no solamente en tareas iniciadas o código escrito.

No indica por sí sola la complejidad de cada funcionalidad: contar funcionalidades no es lo mismo que sumar Story Points.

## RESUMIENDO EL ENFOQUE ÁGIL

- Velocidad: cuánto trabajo se terminó y aceptó.
- Capacidad: cuánto trabajo se estima que puede asumir el equipo en un Sprint.
- RTF: cuántas funcionalidades se encuentran probadas y funcionando.

MÉTRICAS EN KANBAN - ENFOQUE LEAN

También estamos en un contexto de procesos empíricos, pero el foco está puesto en el flujo continuo de trabajo y en el servicio brindado al cliente.

Se busca conocer cuánto tarda una pieza de trabajo en atravesar el proceso, detectar esperas y mejorar el flujo. No interesa solamente cuánto está ocupada una persona, sino qué tan bien fluye el trabajo hasta su entrega.

MÉTRICAS CLAVE

LEAD TIME (TIEMPO DE ENTREGA)

Es la vista del cliente. Mide el tiempo desde que se solicita una pieza de trabajo hasta que se entrega.

Incluye el tiempo que el elemento permanece esperando antes de empezar a trabajarse, además del tiempo de procesamiento y las demás esperas hasta la entrega.

En clase se relaciona con el ritmo de entrega del servicio.

Ejemplo: el cliente solicita una funcionalidad el lunes, el equipo empieza el miércoles y la entrega el viernes. El Lead Time comprende todo el período desde el pedido del lunes hasta la entrega del viernes.

CYCLE TIME (TIEMPO DE CICLO)

Es la vista interna del equipo o del proceso. Mide el tiempo desde que se empieza efectivamente a trabajar sobre una pieza hasta que se termina.

No incluye la espera inicial en el backlog, antes de comenzar a procesarla. Sí puede incluir pausas o bloqueos una vez iniciado el trabajo.
Se relaciona con el ritmo de terminación.

En el ejemplo anterior, el Cycle Time comienza el miércoles y termina el viernes.

DIFERENCIA IMPORTANTE:

Lead Time = desde que el cliente pide hasta que recibe.
Cycle Time = desde que el equipo empieza a trabajar hasta que termina.

Por eso el Lead Time contempla una parte del tiempo que el Cycle Time no incluye: la espera previa al comienzo del trabajo.

TOUCH TIME (TIEMPO DE TRABAJO EFECTIVO)

Es el tiempo en que realmente se trabajó sobre el elemento.

No cuenta el tiempo en colas, buffers, bloqueos o momentos en que nadie está trabajando en esa pieza.

Ejemplo: una tarea puede permanecer tres días en curso, pero si solo se trabajó efectivamente durante un día, su Touch Time es de un día.

EFICIENCIA DEL CICLO DE PROCESO

Compara el tiempo de trabajo efectivo con el tiempo total transcurrido.

Fórmula de la presentación: Eficiencia del ciclo (%) = Touch Time / Elapsed Time × 100

En estos apuntes, Elapsed Time es el tiempo total de entrega considerado para el elemento (Lead Time).

Si gran parte del tiempo total se utiliza trabajando efectivamente, la eficiencia es mayor. Si predominan las esperas, la eficiencia disminuye.

Ejemplo: si una pieza tardó 10 días en entregarse y se trabajó efectivamente 3 días, la eficiencia es 3 / 10 × 100 = 30 %.

El objetivo no es obligar a las personas a trabajar sin pausa, sino identificar esperas y desperdicios del proceso que podrían reducirse.

MÉTRICAS ORIENTADAS A SERVICIO

Además de los tiempos del flujo, la presentación distingue:

- Expectativa de nivel de servicio: lo que el cliente espera recibir.
- Capacidad de nivel de servicio: el nivel que el sistema realmente puede proporcionar.
- Acuerdo de nivel de servicio: el nivel que se acuerda con el cliente.
- Umbral de adecuación del servicio: el límite por debajo del cual el servicio resulta inaceptable para el cliente.

La idea es poder evaluar si el proceso entrega un servicio acorde con las expectativas y compromisos asumidos.

¿Y PARA EL PRODUCTO SOFTWARE, QUÉ MÉTRICAS?
Independientemente del enfoque de gestión, el producto también puede medirse.

Métricas de producto:
TAMAÑO
- Líneas de código.

- Requerimientos.
- Casos de uso.
- Funcionalidades (features).
- Clases.
- Puntos de historia.

DEFECTOS
- Cobertura de pruebas.
- Defectos según severidad.
- Densidad de defectos.

Esto ayuda a distinguir las medidas del producto en sí de las que usamos para seguir la planificación del proyecto o el flujo del proceso.

COMPARACIÓN FINAL DE LOS ENFOQUES

Tradicional: se enfoca en el seguimiento y control de la planificación. Las métricas más representativas son esfuerzo, tiempo, costos y defectos; también se mide el tamaño del producto.

Ágil: se enfoca en inspeccionar los resultados y entregar software funcionando. Sus métricas destacadas son velocidad, capacidad y Running Tested Features (RTF).

Lean / Kanban: se enfoca en el flujo de trabajo y el servicio al cliente. Sus métricas principales son Lead Time, Cycle Time, Touch Time y eficiencia del ciclo de proceso.

CONCLUSIÓN
Las métricas no son un fin en sí mismas. Hay que elegirlas según el contexto, obtenerlas con el menor esfuerzo posible y usarlas para tomar decisiones y realizar mejoras.

Idea importante: MEDIMOS PARA ENTENDER Y MEJORAR, NO PARA JUZGAR.